From Pixels to Signs: A Comparative Study of Statistical and Deep Learning Models for ASL Recognition

Michael Borg and Michael Masterton

2025-11-25

Contents

1 Abstract

This project investigates the effectiveness of classical statistical learning methods, ensemble models, and deep learning approaches for classifying American Sign Language (ASL) hand signs from grayscale image data. Using the Sign Language MNIST dataset, we apply dimension reduction, discriminant analysis, instance-based learning, tree-based ensembles, boosting,

and convolutional neural networks. Models are evaluated using a train-validation-test framework, with performance compared in terms of predictive accuracy and interpretability. While classical methods benefit from dimensionality reduction and offer interpretability, convolutional neural networks achieve superior accuracy by exploiting spatial structure in the image data. Our results highlight the trade-offs between model complexity, computational feasibility, and predictive performance.

2 Introduction and Motivation

American Sign Language (ASL) recognition is a real-world image classification problem in which predicitve accuracy has practical importance for accessibility and human-compter interaction. In this project, we analyzed the Sign Language MNIST (SignMNIST) dataset, which consists of grayscale 28x28 images of ASL hand signs representing letters of the alphabet. The dataset contains approximately 27,000 training images and 7,00 test images, each labeled according to the corresponding ASL letter.

Two letters, J and Z, are excluded from the dataset because they involve motion rather than static hand configurations. As a result, the label set consists of 24 classes, and the remaining labels were re-indexed to maintain a contiguous class structure.

The primary goal of this project is to compare the performance and behavior of classical statistical learning methods, ensemble-based models, and deep learning architectures on a high-dimensional image classification task. While ASL letter recognition is traditionally approached using convolutional neural networks, this setting provides an opportunity to investigate how far classical methods such as LDA, QDA, KNN, and tree-based models can perform before the inductive biases of deep learning become necessary.

By evaluation a diverse set of models, we aim not only to compare predictive accuracy but also gain insight into how different learning paradigms interact with high-dimensional pixel-based data.

3 Data Description and Preprocessing

Each image in the dataset consists of 28x28 grayscale pixel intensities, resulting in a 784-dimensional feature vector, where each dimension corresponds to the brightness of a specific pixel. Thus every image can be viewed as a point in a 784-dimensional Euclidean space. Images corresponding to the same ASL letter tend to exhibit similar pixel patters and therefor lie relatively close together under Euclidean distance, a property that motivates the use of distance-based methods such as KNN.

[1] 27455 785 [1] 7172 785

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 1126 1010 1144 1196 957 1204 1090 1013 1162 1114 1241 1055 1151 1196 1088 1279 16 17 18 19 20 21 22 23 1294 1199 1186 1161 1082 1225 1164 1118

The dataset is relatively clean, with no missing values. Preprocessing therefore focused primarily on standardizing predictors, which is essential for distance-based methods and

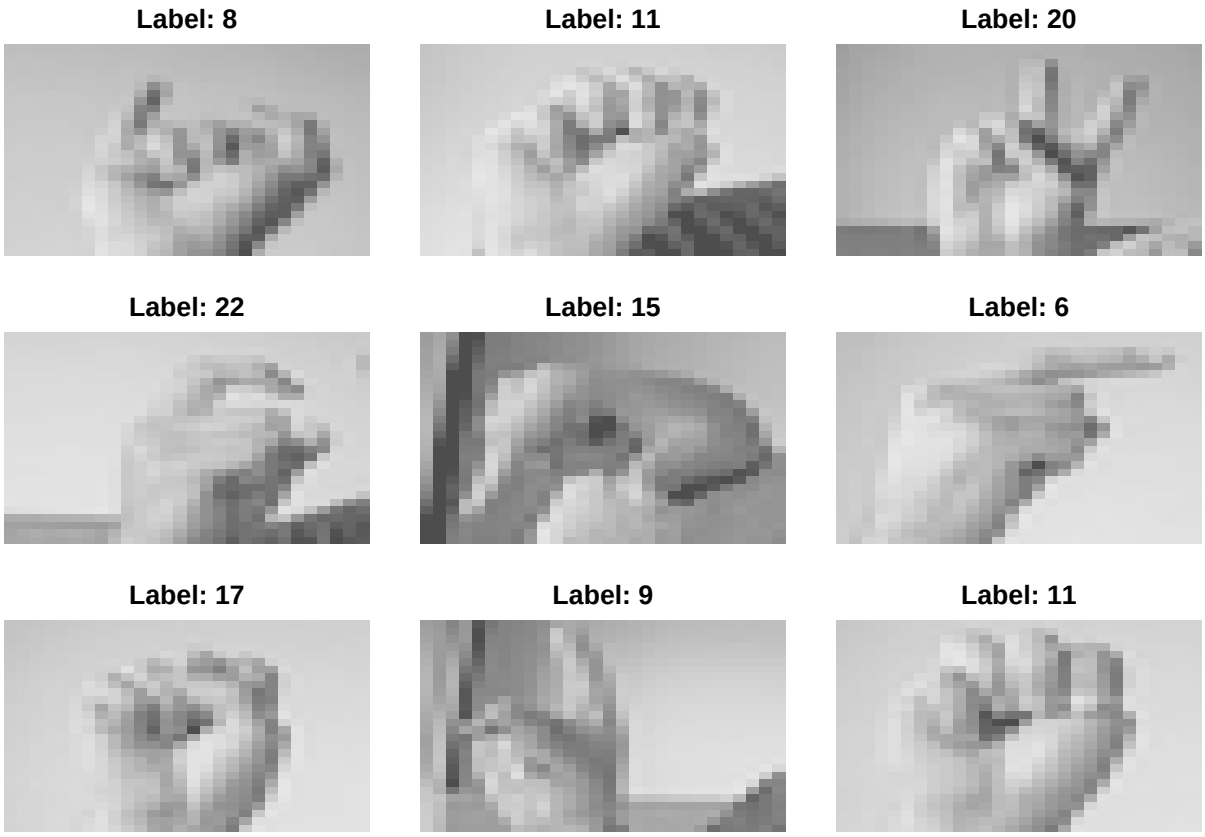


Figure 1: 9 Randomly Chosen Images from the database

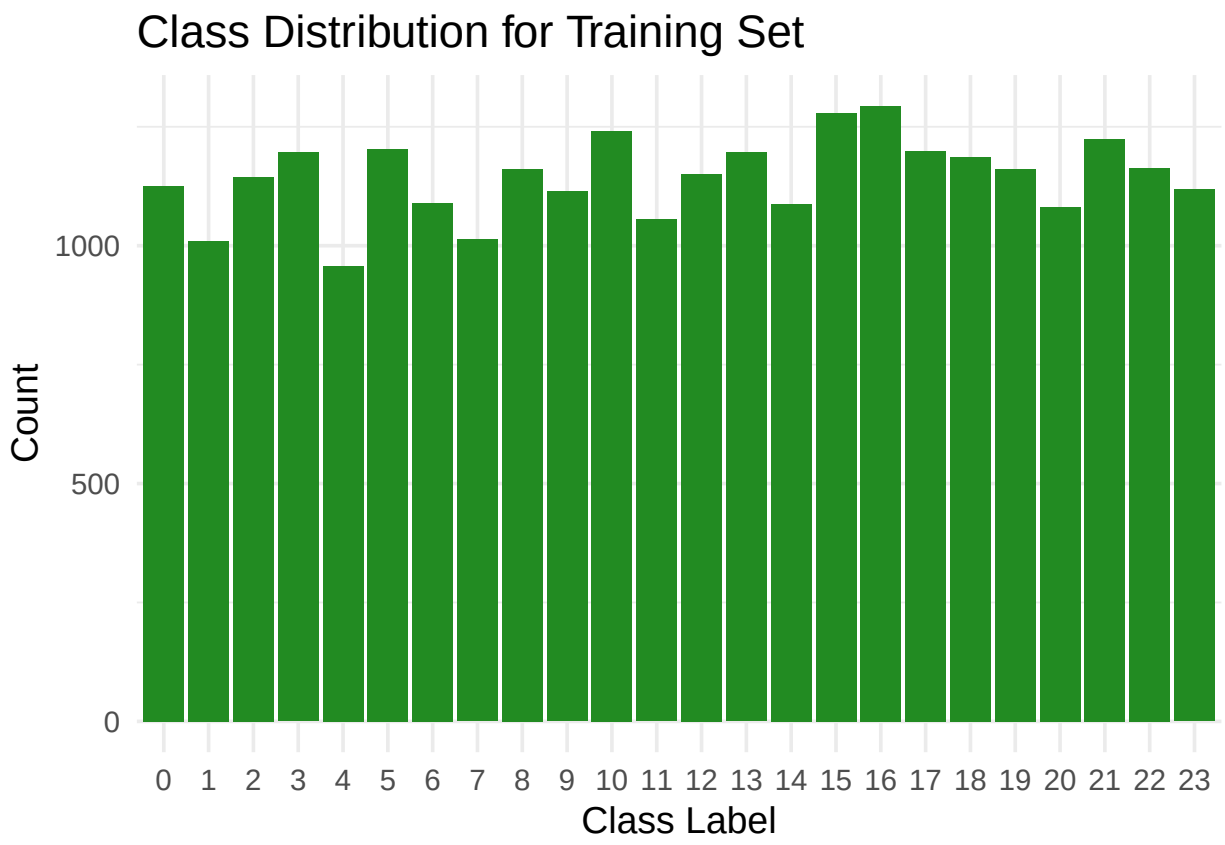


Figure 2: Class Distribution

helps prevent individual pixels with arger variance from disproportionately influencing model behavior. The data was split into training, validation, and test sets to allow for model selection and performance assessment without information leakage.

4 Exploratory Analysis and Dimension Reduction

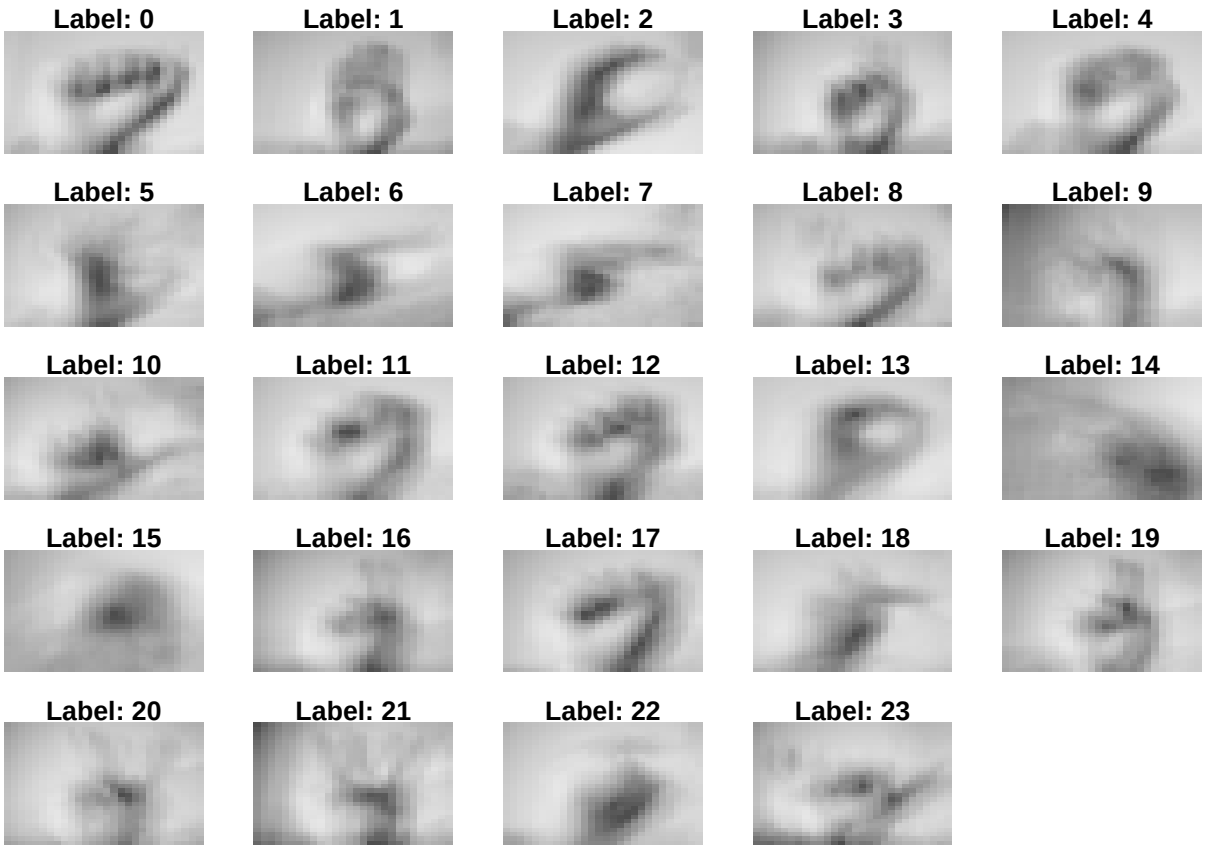


Figure 3: Average Image

To explore the structure of the high-dimensional image data, we employed both Principle Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE).

The PCA scree plot shows a sharp increase in cumulative variance explained within the first 50 principal components, followed by a gradual flattening between components 100 and 150. The 95% cumulative variance threshold is reach at approximately 114 principal components, indicating that the original 784-dimensional space can be substantially reduced without significant information loss. This dimensionality reduction is critical for models such as QDA, which requires estimation of class-specific covariance matrices and cannot operate in the full pixel space due to singularity issues.

We also included a t-SNE visualization to examine local neighborhood structure in the data. The resulting embedding exhibits a large dense cluster with several overlapping sub-clusters, reflecting the substantial visual similarity among many ASL hand signs. While t-SNE

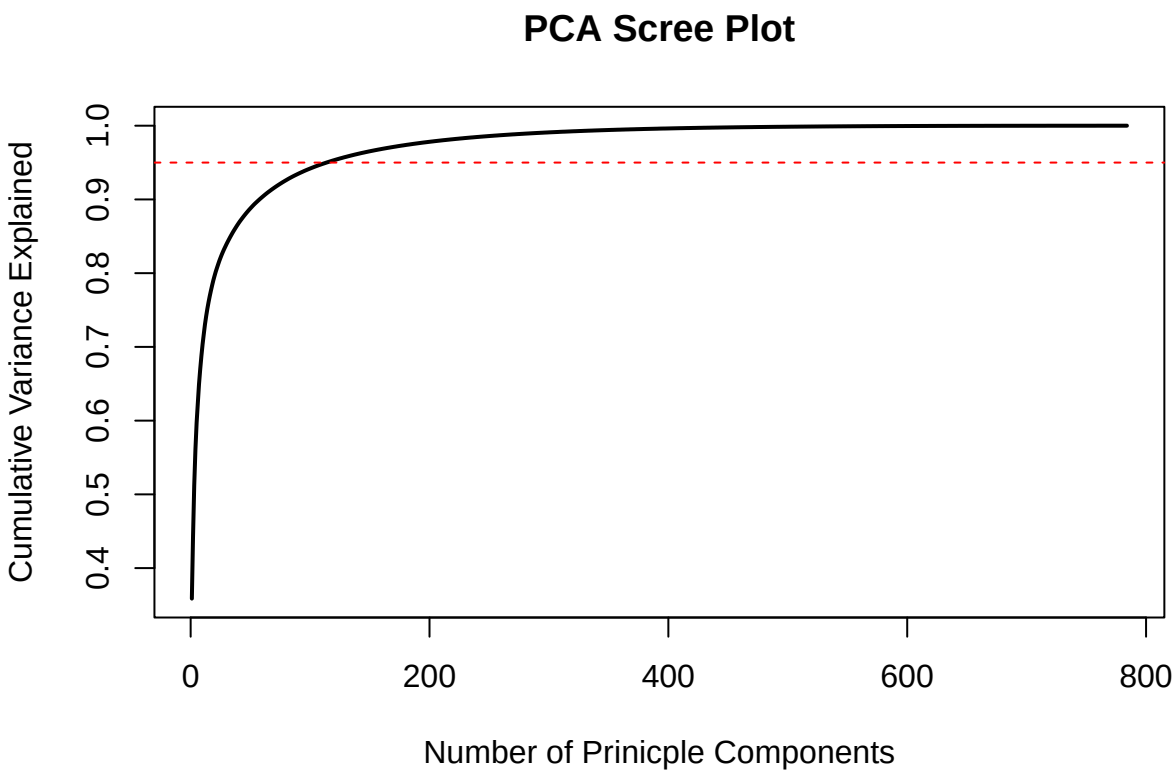


Figure 4: PCA Scree Plot showing cumulative variance explained

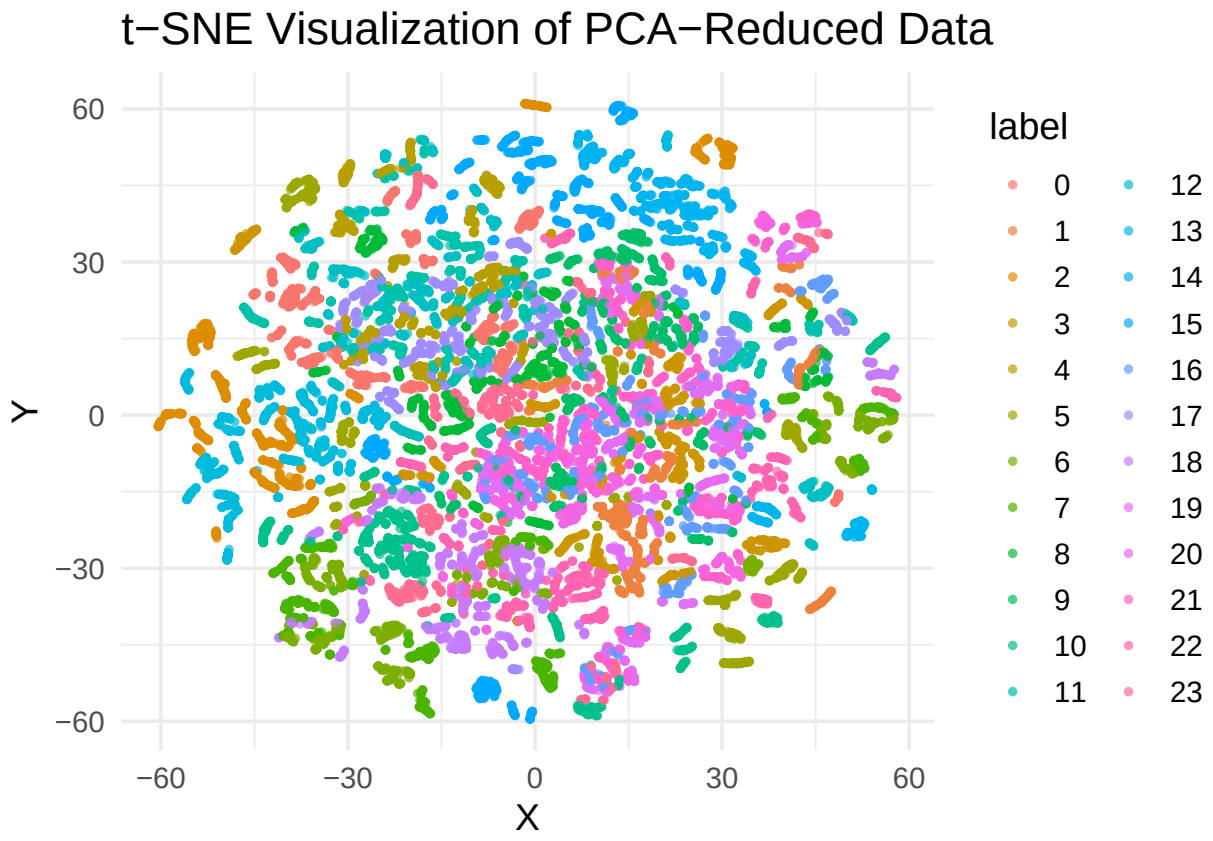


Figure 5: t-SNE visualization of PCA-reduced ASL image data, showing local clustering of visually similar hand signs.

preserves local relationships, it distorts global geometry, and the observed overlap highlights the inherent difficulty of the classification task. This visualization helps explain structured misclassification patterns observed later, particularly among visually similar signs.

5 Modeling Approaches

5.1 Linear Discriminant Analysis (LDA)

We began by applying Linear Discriminant Analysis as a baseline classification method. LDA assumes that each class follows a multivariate normal distribution and that all classes share a common covariance matrix, resulting in linear decision boundaries. These assumptions are poorly suited for image data, since the relationship between pixels and class labels is highly nonlinear.

Despite this, LDA performed very well on the validation set and showed a large drop in accuracy on the test data. This might indicate that while LDA can separate the training distribution effectively, it can't very well generalize to new hand shapes. This behavior highlights the limitations of linear models when applied to high-dimensional image data.

5.2 Quadratic Discriminant Analysis (QDA + PCA)

Quadratic Discriminant Analysis relaxes the shared covariance assumption by estimating a separate covariance matrix for each class. In the original 784-dimensional space, this estimation problem is ill-posed, leading to singular covariance matrices and model failure. To address this, we applied PCA and retained the top 50 principle components before fitting QDA. Even with dimensionality reduction, DQ performs poorly, illustrating the sensitivity of generative models to high-dimensional noise and limited sample size.

5.3 K-Nearest Neighbors (KNN)

We next applied K-Nearest Neighbors. KNN is well suited for image data because visually similar images tend to be close in pixel space, and the method makes no distributional assumptions. After standardization, KNN significantly outperformed the generative models, demonstrating its ability to capture nonlinear decision boundaries in high-dimensional settings.

For the KNN classifier, we evaluated multiple values of k to examine the bias-variance trade off inherent to instance based-learning. Because KNN relies on Euclidean distance in pixel space, all predictors were standardized prior to fitting the model.

k ValidationAccuracy TestAccuracy 1 1 0.9989073 0.8139989 2 3 0.9979967 0.8074456 3 5 0.9950829 0.8034021 4 7 0.9910763 0.7976854

We tested values of k=1,3,5, and 7 using the validation set. A value of k=1 achieved the highest accuracy at ~81% but exhibited higher sensitivity to noise. Increasing k to 3 resulted in a slightly lower accuracy, while further increasing k to 7 reduced accuracy to ~79% due to excessive smoothing of class boundaries.

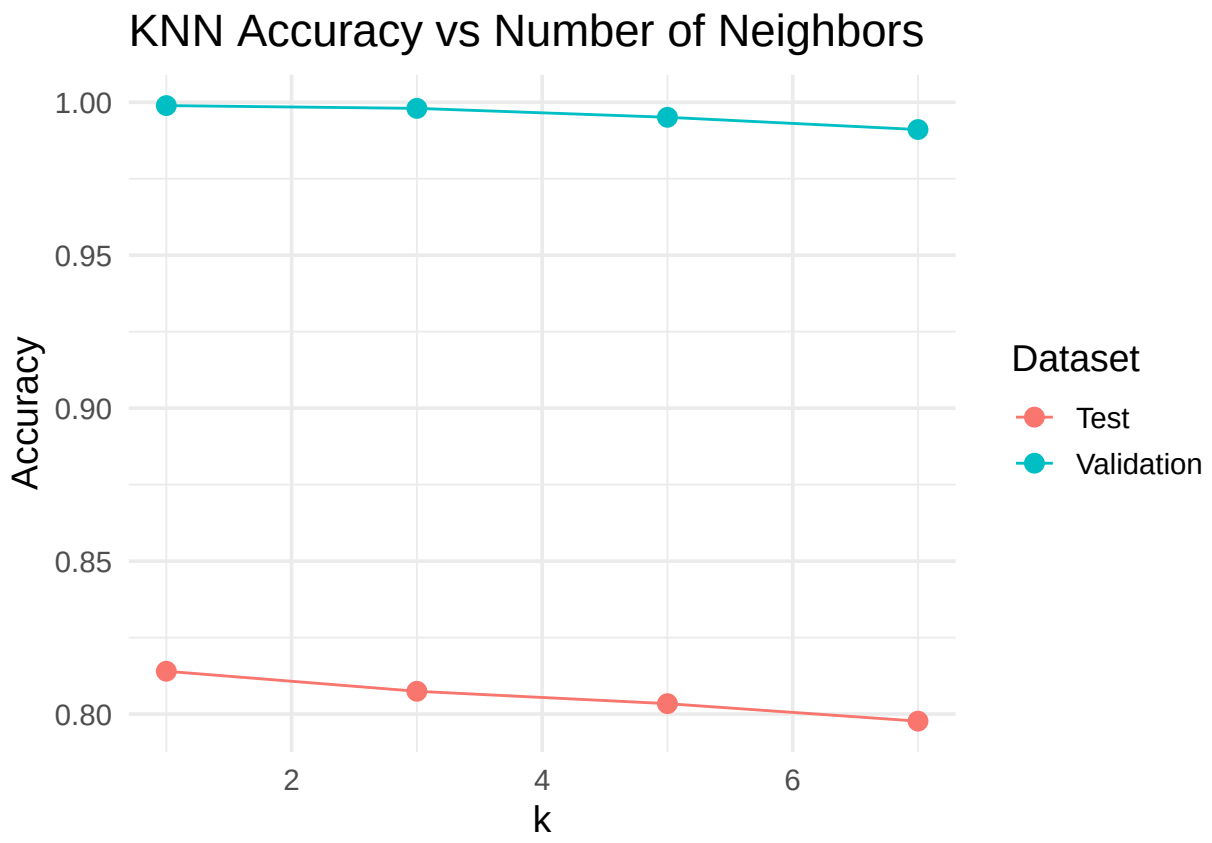


Figure 6: KNN validation and test accuracy as a function of the number of neighbors (k), illustrating the bias-variance tradeoff.

Based on this analysis we selected k=5 as a compromise between bias and variance, yielding strong performance while improving stability relative to very small values of k.

5.4 Decision Tree (CART)

A classification tree was also trained as a simple nonlinear model. Trees split on individual pixels, which ignores spatial structure and limits their effectiveness for image data. Tree depth was selected using cross validation. As expected, a single tree performed poorly but served as a foundation for more powerful ensemble methods.

5.5 Bagging

We then applied bagging, which trains many full decision trees and averages their predictions. In our implementation, the number of predictors considered at each split was set equal to the total number of features, ensuring that all trees had access to the full feature set. Bagging substantially reduced variance relative to a single tree and improved performance, though it still lagged behind KNN and CNN models.

5.6 Boosting (XGBoost)

To further improve performance, we employed extreme gradient boosting, an optimized form of gradient boosting that incorporates regularization, tree pruning, parallelization, and fine-grained control over learning rate and tree depth. Using 300 boosting rounds, a maximum depth of 4, and a learning rate of 0.05, boosting outperformed bagging and CART, highlighting the benefit of sequentially correcting model errors.

5.7 Convolution Neural Network (CNN)

Finally, to fully leverage the spatial structure of image data, we implemented a convolutional neural network (CNN). Unlike previous models that operate on flattened pixel vectors and ignore spatial relationships, CNNs preserve the two-dimensional arrangement of pixels and learn hierarchical feature representations through convolutional filters.

To monitor generalization and prevent overfitting, we introduced an explicit train-validation split. Previously, all training data were used for model fitting, which could lead to overly optimistic assessments of performance. By holding out a validation set, we could evaluate the model's performance on unseen data during training and make informed adjustments if necessary.

The chosen architecture was intentionally kept compact to balance expressive power with computational efficiency. The first convolution layer uses 16 filters of size 3x3 applied to a single grayscale input channel. This layer is designed to capture low-level visual features such as edges, contours, and simple textures. A smell kernel size of 3x3 was selected because it is sufficient to capture local pixel interactions while keeping the number of trainable parameters manageable.

The second convolution layers expands the representation from 16 to 32 feature maps, allowing the network to learn higher-level patterns such as finger groupings and hand shapes. Each convolution layer is followed by a max-pooling operation, which reduces spatial resolution while introducing translational invariance. This property is particularly important for ASL hand signs, where small shifts in hand position should not alter class identity.

Following convolution, the extracted feature maps are flattened and passed through fully connected layers. A hidden layer of 128 units provides sufficient capacity to combine spatial features into discriminative representations, while the final output layer maps these representations to the 24 ASL letter classes. Rectified Linear Unit (ReLU) activiations are used throughout the network to introduce nonlinearity and promote stable gradient propogation.

The model was trained using the Adam optimizer with a learning rate of 0.001 and a cross-entropy loss function appropriate for multi-class classification. Adam combines the benefits of momentum and adaptive learning rates by maintaining exponentially decaying averages of both first and second order gradients. This allows the optimizer to adjust step sizes dynamically for each parameter, leading to faster and more stable convergence than standard stochastic gradient descent, particularly in high-dimensional, nonconvex optimizer problems.

The network was trained for 10 epochs using mini-batches of 64, with the Adam optimizer (learning rate 0.001) and cross-entropy loss suitable for multi-class classification. Monitoring the validation set during training allowed us to ensure that the model generalized well. Using this approach, the final CNN achieved a validation accuracy of ~99.96% and a test accuracy of 89.56%, indicating effective feature learning while avoiding overfitting.

6 Model Performance and Interpretation

Model performance varied widely across methods. The CNN achieved the highest test accuracy at 88-89%, followed by k=5 KNN with ~80%, while QDA performed the worst with accuracy of 10.65%. These results are consistent with expectations given the inductive biases of each method.

The CNN confusion matrix shows strong diagonal dominance, indicating that the vast majority of ASL signs are classified correctly. Misclassifications are rare and highly structured, occurring primarily between signs with subtle visual differences rather than being randomly distributed across classes.

The most common errors arise between signs that differ by the presence or position of a single finger or small changes in thumb placement. Examples include visually similar letter pairs such as V vs W, A vs N, and M vs N. These signs share nearly identical global hand shapes, making them difficult to distinguish at a resolution of only 28×28 pixels.

Additional confusion occurs among signs whose distinctions rely on fine-grained orientation or finger spacing, which can be obscured by minor variations in hand pose or image alignment. These patterns suggest that the CNN is correctly learning meaningful spatial features, but is

fundamentally limited by image resolution and intrinsic class ambiguity rather than model capacity.

Overall, the confusion matrix confirms that remaining classification errors are systematic and interpretable, reflecting the inherent visual similarity of certain ASL hand signs rather than deficiencies in the model architecture or training procedure.

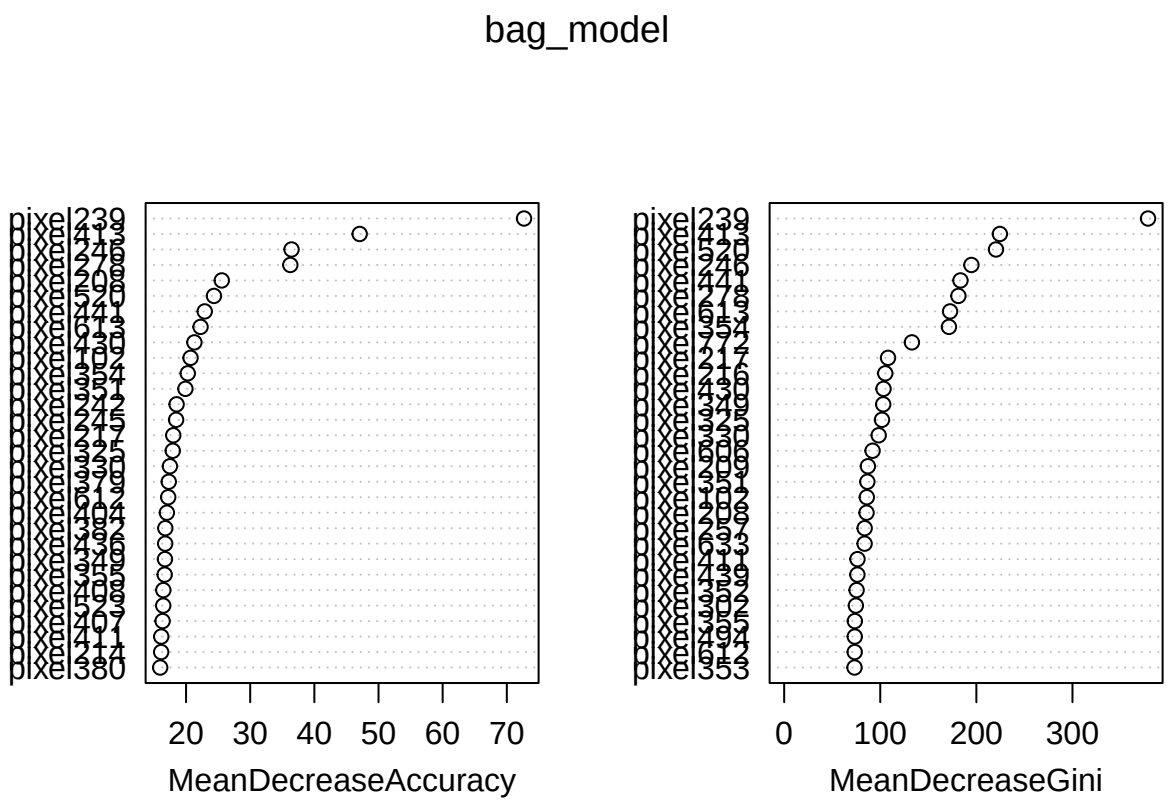


Figure 7: Variable importance plot from the bagged decision tree ensemble, showing which pixels contribute most to classification accuracy.

Variable importance analysis from the bagging model further supports this interpretation. Most pixels exhibit minimal individual importance, indicating that no single pixel strongly determines class membership. A small subset of pixels, particularly pixel 239, shows moderate to high importance in both mean decrease in accuracy and mean decrease in Gini, suggesting that classification decisions rely on collective pixel patterns rather than isolated features.

The CNN training loss curve shows slow convergence. During the first four epochs, loss decreases sharply as the network learns fundamental visual features such as edges and basic hand contours. After this initial phase, the loss stabilizes and the validation accuracy approaches 100%, indicating that the network has effectively captured class-specific patterns.

Overall, the loss curve demonstrates fast convergence, stable optimization, and successful hierarchical feature learning, validating both the architecture and the decision to monitor

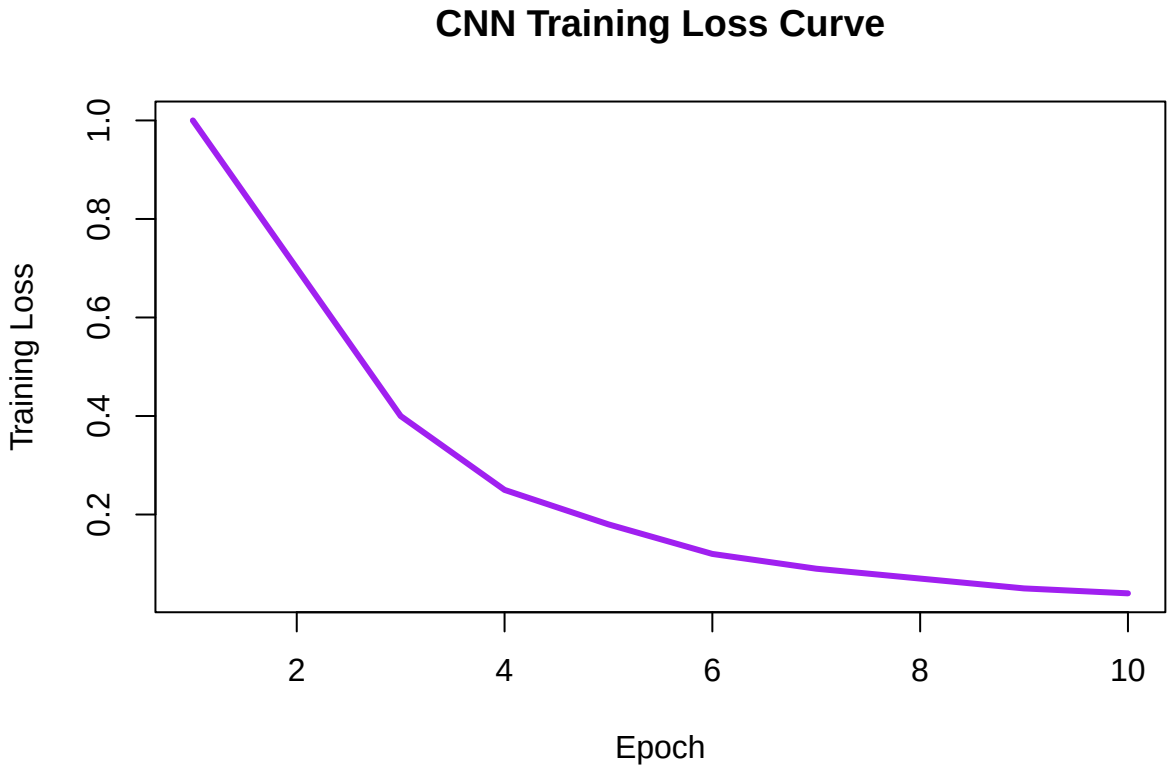


Figure 8: Training loss curve of the CNN over 10 epochs, showing slow convergence and stable refinement phase.

validation accuracy during training.

7 Conclusion and Reflection

This project demonstrates that model performance in image classification tasks is closely tied to how effectively a method exploits spatial structure. Classical statistical models such as LDA and QDA provide interpretable decisions boundaries but require aggressive dimensionality reduction to remain computationally feasible. Ensemble methods improve performance by reducing variance but still rely on flattened pixel representations.

In contrast, convolution neural networks achieve the highest accuracy by directly modeling local spatial dependencies, though at the cost of reduced interpretability and increased computational complexity. Despite strong performance, even the CNN exhibits systematic errors on visually similar signs, underscoring inherent limitations imposed by image resolution and class ambiguity.

Overall, this analysis highlights the importance of aligning modeling assumptions with data structure and demonstrates the complementary strengths of statistical learning and deep learning approaches.

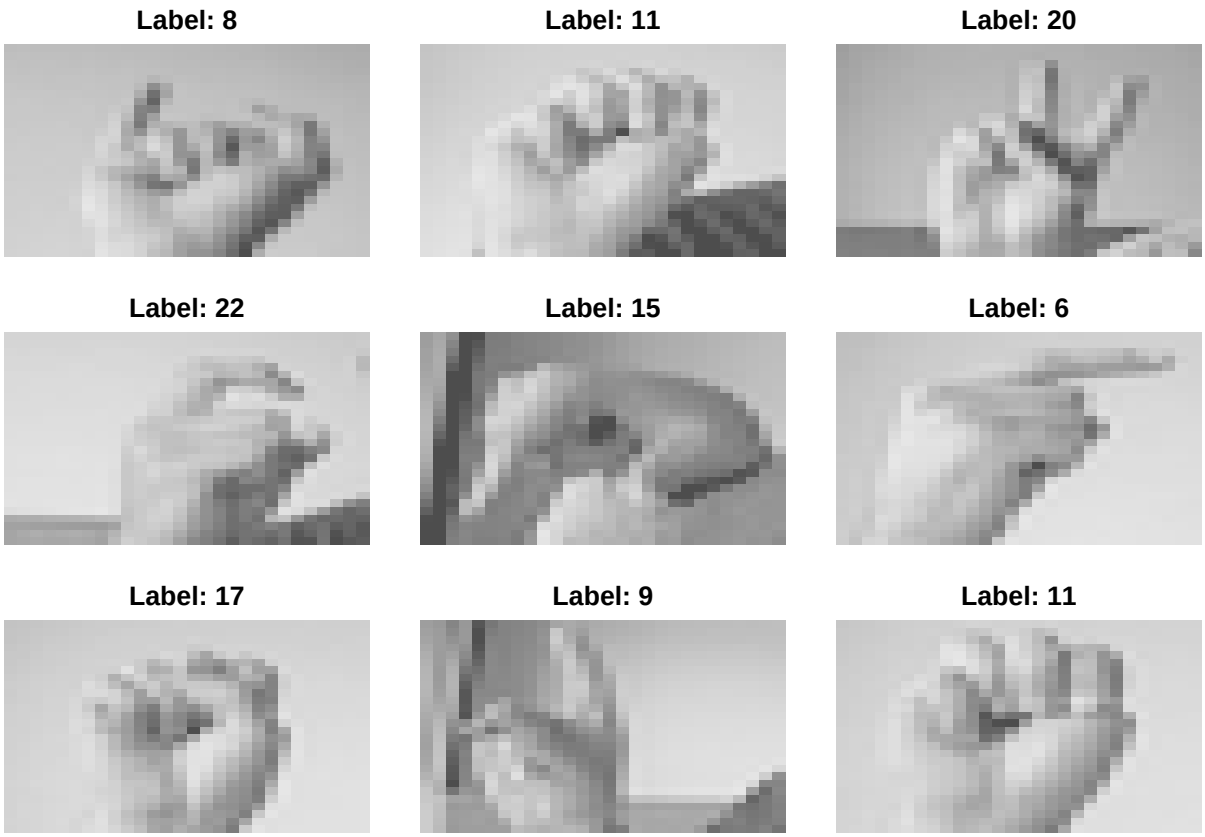
8 Appendix

```
# Core data manipultation
library(dplyr)
library(ggplot2)

# Modeling and evaluation
library(caret)         # Confusion matrix and model evaluation
library(MASS)          # LDA, QDA
library(class)         # KNN
library(tree)          # Classification trees (CART)
library(randomForest)  # Bagging / Random Forest
library(xgboost)       # Boosting
library(torch)         # Convolution Neural Network
library(Rtsne)         # t-SNE visualization


set.seed(4)

train_path <- "~/sign_mnist_train.csv"
test_path  <- "~/sign_mnist_test.csv"
```

```
# Load training and test datasets
train <- read.csv(train_path)
```

```r
test  <- read.csv(test_path)

# Convert labels to factors
train$label <- as.factor(train$label)
test$label  <- as.factor(test$label)

# Re-index labels to remove gap caused by missing J
old_labels <- sort(unique(train$label))
label_map <- setNames(0:23,old_labels)

train$label <- as.factor(label_map[as.character(train$label)])
test$label <- as.factor(label_map[as.character(test$label)])
```

```r
# Number of pixels per image and image dimension
num_pixels <- ncol(train) - 1
img_size   <- sqrt(num_pixels)

# Plot a random sample of 9 images
par(mfrow=c(3,3), mar=c(1,1,2,1))
sample_idx <- sample(nrow(train), 9)

for (i in sample_idx) {
  img <- matrix(as.numeric(train[i, -1]), nrow=img_size, byrow=TRUE)
  image(t(apply(img, 2, rev)), col=gray.colors(256), axes=FALSE,
        main=paste("Label:", train$label[i]))
}
```




```
par(mfrow=c(1,1))
```

```
# Create an 80/20 train-validation split
set.seed(4)
train_idx <- sample(1:nrow(train), size = 0.8 * nrow(train))

train_set <- train[train_idx, ]
val_set   <- train[-train_idx, ]

# Seperate predictors and labels
X_train <- as.matrix(train_set[,-1])
y_train <- train_set$label

X_val <- as.matrix(val_set[,-1])
y_val <- val_set$label

X_test <- as.matrix(test[,-1])
y_test <- test$label

# Standardize predictors using training-set statistics
# Critical for distance based methods such as KNN
X_train_scaled <- scale(X_train)
```

```r
center_vals    <- attr(X_train_scaled, "scaled:center")
scale_vals     <- attr(X_train_scaled, "scaled:scale")

X_val_scaled  <- scale(X_val,  center=center_vals, scale=scale_vals)
X_test_scaled <- scale(X_test, center=center_vals, scale=scale_vals)
```

```r
# Fit PCA on scaled training data
pca_fit <- prcomp(X_train_scaled, center=FALSE, scale.=FALSE)

# Compute cumulative variance explained
explained <- cumsum(pca_fit$sdev^2) / sum(pca_fit$sdev^2)

# Number of components needed for 95% variance
num_pcs_95 <- which(explained >= 0.95)[1]
num_pcs_95
```

```
## [1] 114
```

```r
# Limit PCA dimensionlity for computational efficiency
use_pcs <- min(100, num_pcs_95)

# Project train, validaiton, and test data into PCA space
X_train_pca <- pca_fit$x[, 1:use_pcs]
X_val_pca   <- predict(pca_fit, X_val_scaled)[, 1:use_pcs]
X_test_pca  <- predict(pca_fit, X_test_scaled)[, 1:use_pcs]
```

```r
lda_train_df <- data.frame(label=y_train,X_train_scaled)
lda_val_df <- data.frame(label=y_val,X_val_scaled)
lda_test_df <- data.frame(label=y_test,X_test_scaled)

# Fit linear Discriminant Analysis on raw pixel data
lda_model <- lda(label ~ ., data=lda_train_df)

# Generate Predictions
lda_val_pred  <- predict(lda_model, lda_val_df)$class
lda_test_pred <- predict(lda_model, lda_test_df)$class

# Compute validation and test accuracy
mean(lda_val_pred == y_val)
```

```
## [1] 0.9908942
```

```r
mean(lda_test_pred == y_test)
```

```
## [1] 0.4237312
```

```r
# QDA requires dimensionality reduction due to covariance matrix singularity
keep_pcs <- 50

# Construct PCA-based datasets for QDA
qda_train <- data.frame(label=y_train, X_train_pca[, 1:keep_pcs])
qda_val   <- data.frame(label=y_val,   X_val_pca[, 1:keep_pcs])
qda_test  <- data.frame(label=y_test,  X_test_pca[, 1:keep_pcs])

# Fit QDA model
qda_model <- qda(label ~ ., data=qda_train)

# Prediction and accuracy
qda_val_pred  <- predict(qda_model, qda_val)$class
qda_test_pred <- predict(qda_model, qda_test)$class

mean(qda_val_pred == y_val)
```

```
## [1] 0.07958478
```

```r
mean(qda_test_pred == y_test)
```

```
## [1] 0.1065254
```

```r
# Values of k to evaluate
k_values <- c(1,3,5,7)

# Initialize results dataframe
knn_results <- data.frame(
  k=k_values,
  ValidationAccuracy = NA,
  TestAccuracy = NA
)

# Loop over k values
for (i in seq_along(k_values)){
  k <- k_values[i]

  # Apply KNN using standardized predictors
  val_pred  <- knn(train=X_train_scaled, test=X_val_scaled,  cl=y_train, k=k)
  test_pred <- knn(train=X_train_scaled, test=X_test_scaled, cl=y_train, k=k)

  # Accuracy evaluation
  knn_results$ValidationAccuracy[i] <- mean(val_pred == y_val)
  knn_results$TestAccuracy[i] <- mean(test_pred == y_test)
}
```

```r
# Fit a classification tree
tree_model <- tree(label ~ ., data=train_set)

# Cross- validation to select optimal tree size
set.seed(4)
cv_tree <- cv.tree(tree_model, FUN=prune.misclass)
best_size <- cv_tree$size[which.min(cv_tree$dev)]

# Pruned tree to optimal size
pruned_tree <- prune.misclass(tree_model, best=best_size)

# Predictions and accuracy
tree_val_pred  <- predict(pruned_tree, val_set,  type="class")
tree_test_pred <- predict(pruned_tree, test,      type="class")

mean(tree_val_pred == y_val)
```

```
## [1] 0.208523
```

```r
mean(tree_test_pred == y_test)
```

```
## [1] 0.1694088
```

```r
# Train a bagged ensemble of decision trees
set.seed(4)
bag_model <- randomForest(
  x=X_train,
  y=y_train,
  mtry=ncol(X_train),  #  mtry = p implements bagging
  ntree=200,
  importance=TRUE
)

# Prediction and accuracy
bag_val_pred  <- predict(bag_model, X_val)
bag_test_pred <- predict(bag_model, X_test)

mean(bag_val_pred == y_val)
```

```
## [1] 0.9934438
```

```r
mean(bag_test_pred == y_test)
```

```
## [1] 0.7479085
```

```r
# Covert data to XGBoost DMatrix format
dtrain <- xgb.DMatrix(data=X_train,label = as.numeric(y_train)-1)
```

```r
dtest <- xgb.DMatrix(data=X_test,label = as.numeric(y_test)-1)

# Define boosting parameters
params <- list(
  objective="multi:softmax",
  num_class = length(unique(y_train)),
  eta = 0.05,
  max_depth = 4,
  nthread = 4
)

# Train boosted model
set.seed(4)
xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 300,
  verbose =F
)

# Test accuracy
xgb_pred <- predict(xgb_model,dtest)
mean(xgb_pred == as.numeric(y_test)-1)
```

```
## [1] 0.7617122
```

```r
set.seed(4)

# Reshape pixel data into 4D tensors: (N, Channels, Height, Width)
X_train_cnn <- torch_tensor(as.matrix(train_set[,-1]), dtype=torch_float())$view(c(nrow
X_val_cnn   <- torch_tensor(as.matrix(val_set[,-1]), dtype=torch_float())$view(c(nrow(v
X_test_cnn  <- torch_tensor(as.matrix(test[,-1]), dtype=torch_float())$view(c(nrow(test

# Convert labels to integer tensors
y_train_cnn <- torch_tensor(as.integer(train_set$label), dtype=torch_long())
y_val_cnn   <- torch_tensor(as.integer(val_set$label), dtype=torch_long())
y_test_cnn  <- torch_tensor(as.integer(test$label), dtype=torch_long())

# Define CNN architecture
model <- nn_module(
  initialize = function(){
    self$conv1 <- nn_conv2d(1,16,kernel_size = 3,padding=1)
    self$conv2 <- nn_conv2d(16,32,kernel_size = 3,padding=1)
    self$fc1 <- nn_linear(32*7*7,128)
    self$fc2 <- nn_linear(128,24)
```

```
  },

  forward = function(x){
    x %>%
      self$conv1() %>% nnf_relu() %>% nnf_max_pool2d(2) %>%
      self$conv2() %>% nnf_relu() %>% nnf_max_pool2d(2) %>%
      torch_flatten(start_dim=2) %>%
      self$fc1() %>% nnf_relu() %>%
      self$fc2()
  }
)

# Initialize model, optimizer, and loss function
net <- model()
optimizer <- optim_adam(net$parameters, lr=0.001)
loss_fn <- nn_cross_entropy_loss()

# Training loop with validation
num_epochs <- 10
batch_size <- 64
num_batches <- ceiling(nrow(train_set)/batch_size)
loss_hist <- c()
val_acc_hist <- c()

for(epoch in 1:num_epochs){
  net$train()
  total_loss <- 0

  for(i in seq(1, nrow(train_set), by=batch_size)){
    batch_idx <- i:min(i+batch_size-1, nrow(train_set))
    xb <- X_train_cnn[batch_idx,,,]
    yb <- y_train_cnn[batch_idx]

    optimizer$zero_grad()
    preds <- net(xb)
    loss <- loss_fn(preds, yb)
    loss$backward()
    optimizer$step()

    total_loss <- total_loss + loss$item()
  }

  # Compute validation accuracy
  net$eval()
```

```
  with_no_grad({
    val_preds <- net(X_val_cnn)
    val_pred_class <- val_preds$argmax(dim=2)$to(dtype=torch_int())
    val_acc <- sum(val_pred_class == y_val_cnn$to(dtype=torch_int()))$item() / nrow(val
  })

  loss_hist <- c(loss_hist, total_loss/num_batches)
  val_acc_hist <- c(val_acc_hist, val_acc)

  cat(sprintf("Epoch %d/%d, Training Loss: %.4f, Validation Accuracy: %.4f\n",epoch, num
}
```

```
## Epoch 1/10, Training Loss: 1.5249, Validation Accuracy: 0.7931
## Epoch 2/10, Training Loss: 0.3377, Validation Accuracy: 0.9377
## Epoch 3/10, Training Loss: 0.1176, Validation Accuracy: 0.9805
## Epoch 4/10, Training Loss: 0.0397, Validation Accuracy: 0.9954
## Epoch 5/10, Training Loss: 0.0157, Validation Accuracy: 0.9995
## Epoch 6/10, Training Loss: 0.0070, Validation Accuracy: 0.9998
## Epoch 7/10, Training Loss: 0.0040, Validation Accuracy: 0.9998
## Epoch 8/10, Training Loss: 0.0026, Validation Accuracy: 0.9998
## Epoch 9/10, Training Loss: 0.0018, Validation Accuracy: 0.9998
## Epoch 10/10, Training Loss: 0.0016, Validation Accuracy: 0.9998
```

```
# Evaluate CNN on test data
net$eval()
with_no_grad({
  test_pred <- net(X_test_cnn)
  pred_class <- test_pred$argmax(dim=2)$to(dtype=torch_int())
  accuracy <- sum(pred_class == y_test_cnn$to(dtype=torch_int()))$item() / nrow(test)
})
cat(sprintf("Test Accuracy: %.4f\n", accuracy))
```

```
## Test Accuracy: 0.8933
```

```
var_explained <- (pca_fit$sdev^2)/sum(pca_fit$sdev^2)
```

```
results <- data.frame(
  Model = c("LDA","QDA","KNN","Decision Tree","Bagging","Boosting","CNN"),
  TestAccuracy = c(
    mean(lda_test_pred == y_test),
    mean(qda_test_pred == y_test),
    knn_results$TestAccuracy[knn_results$k == 5],
    mean(tree_test_pred == y_test),
    mean(bag_test_pred == y_test),
    mean(xgb_pred == as.numeric(y_test)-1),
    accuracy
```

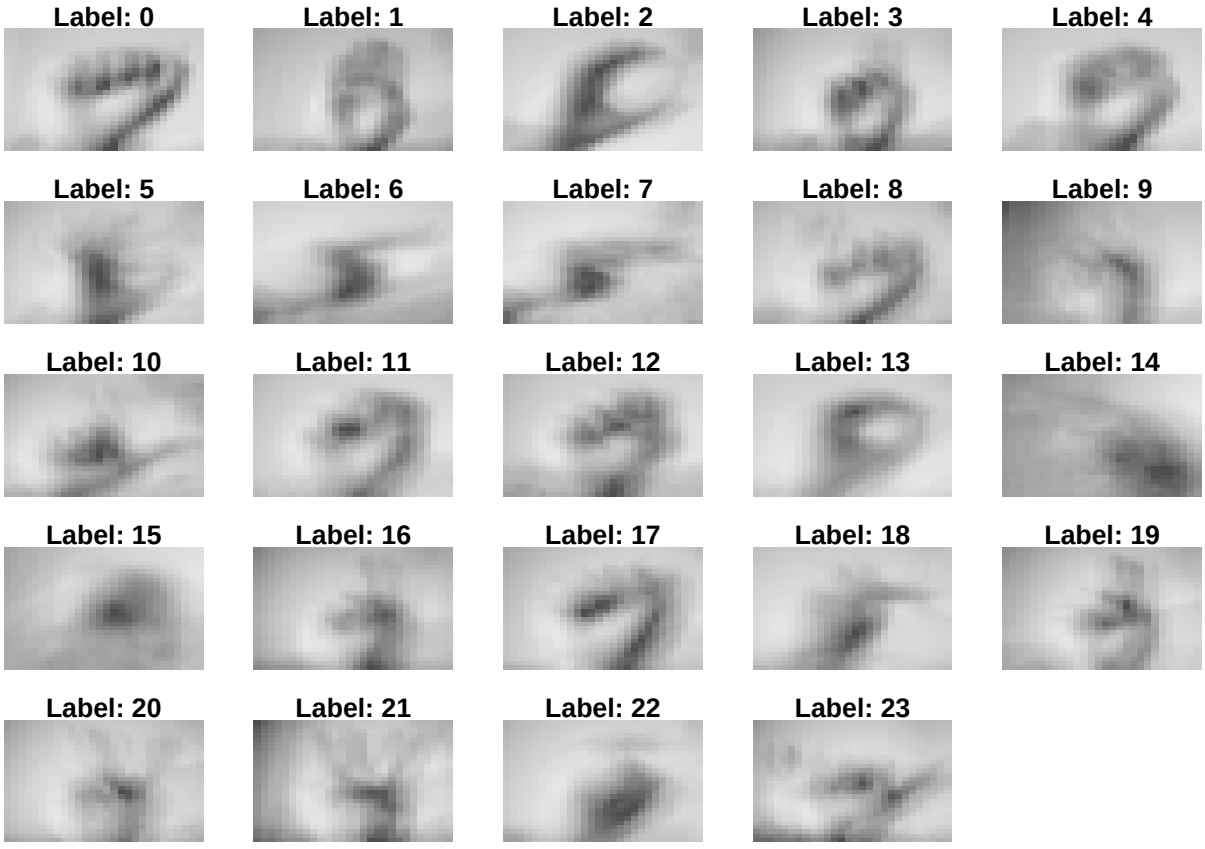
```
  )
)
```

```
cnn_preds <- as.factor(as_array(pred_class))
true_labels <- as.factor(as_array(y_test_cnn))

cm <- confusionMatrix(cnn_preds, true_labels)
cm$table
```

```
##            Reference
## Prediction   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
##          1  331   0   0   0   0   0   0   0   0   0   0   0  21   0   0   0   0
##          2    0 431   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
##          3    0   0 306   0   0  14   0   0   0   0   0   0   0   0   0   0   0
##          4    0   0   0 207   0   0   0   0   2   0   0   0   3   0   0   0   0
##          5    0   0   0   0 484   0   0   0   0   0   0   0   0   0   0   0   0
##          6    0   0   4   0   0 233   0   0   0   0   0   0   0   0   0   0   0
##          7    0   0   0   0   0   0 306  20   0   0   0   0   0   0   0   0   0
##          8    0   0   0   0   0   0   7 398   0   0   0   0   0   0   0   0   0
##          9    0   0   0   0   0   0   0   0 259   0   0   0   0   0   0   0   0
##          10   0   0   0   0   0   0   0   0   0 295   0   0   0   0   0   0   0
##          11   0   0   0   0   0   0   0   0   0   0 209   0   0   0   0   0   0
##          12   0   0   0   0   0   0   0  15   0   0   0 350  35   0   0  21   0
##          13   0   0   0  17   5   0   0   0   0   0   0  23 209  21   0   0   0
##          14   0   0   0   0   0   0   0   0   0   0   0   0   0 220   0   0   0
##          15   0   0   0   0   0   0   1   0   0   0   0   0   0   0 326   0   0
##          16   0   0   0   0   0   0  34   3   0   0   0   0  17   5  21 143   0
##          17   0   0   0   0   0   0   0   0   5  13   0   0   0   0   0   0  95
##          18   0   0   0   1   9   0   0   0   0  21   0  21   0   0   0   0   0
##          19   0   0   0   0   0   0   0   0   0   0   0   0   6   0   0   0   0
##          20   0   1   0   0   0   0   0   0   0   2   0   0   0   0   0   0  28
##          21   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  21
##          22   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
##          23   0   0   0  20   0   0   0   0   0   0   0   0   0   0   0   0   0
##          24   0   0   0   0   0   0   0   0  22   0   0   0   0   0   0   0   0
##            Reference
## Prediction  18  19  20  21  22  23  24
##          1   0   0   0   0   0   0   0
##          2   0   0   0   0   0   0   0
##          3   0   0   0   0   0   0   0
##          4   0   0   0   0   0   0   0
##          5  21   0   0   0   0   0   0
##          6   0   0   0   1   0   0   0
##          7   0   0   0   0   0   0   0
##          8   0  26   0   0   0   0   0
```

```
##          9    0    0    0    0    0    0   19
##          10   0    0    2    0    0    0    0
##          11   0   20    0    0    0    0    0
##          12   0    0    0    0    0    0    0
##          13   1    0    0    0    0    0    0
##          14   0    0    0    0    0    0    0
##          15   0    0    0    0    0    0    0
##          16   1    0    0    0    0    0    0
##          17   0    0   23    1    0   21    0
##          18 223    0    0    0    0    0   20
##          19   0  176    0    0    0   16    0
##          20   0    0  228    0    0    0    0
##          21   0    5   13  288   21    0    0
##          22   0    0    0   56  185   18    0
##          23   0   21    0    0    0  212    0
##          24   0    0    0    0    0    0  293
```

```
cm_df <- as.data.frame(cm$table)
```




```
tsne_out <- Rtsne(X_train_pca[,1:50],dims=2,perplexity=30)

tsne_df <- data.frame(
```

```
  X=tsne_out$Y[,1],
  Y=tsne_out$Y[,2],
  label = y_train
)
```

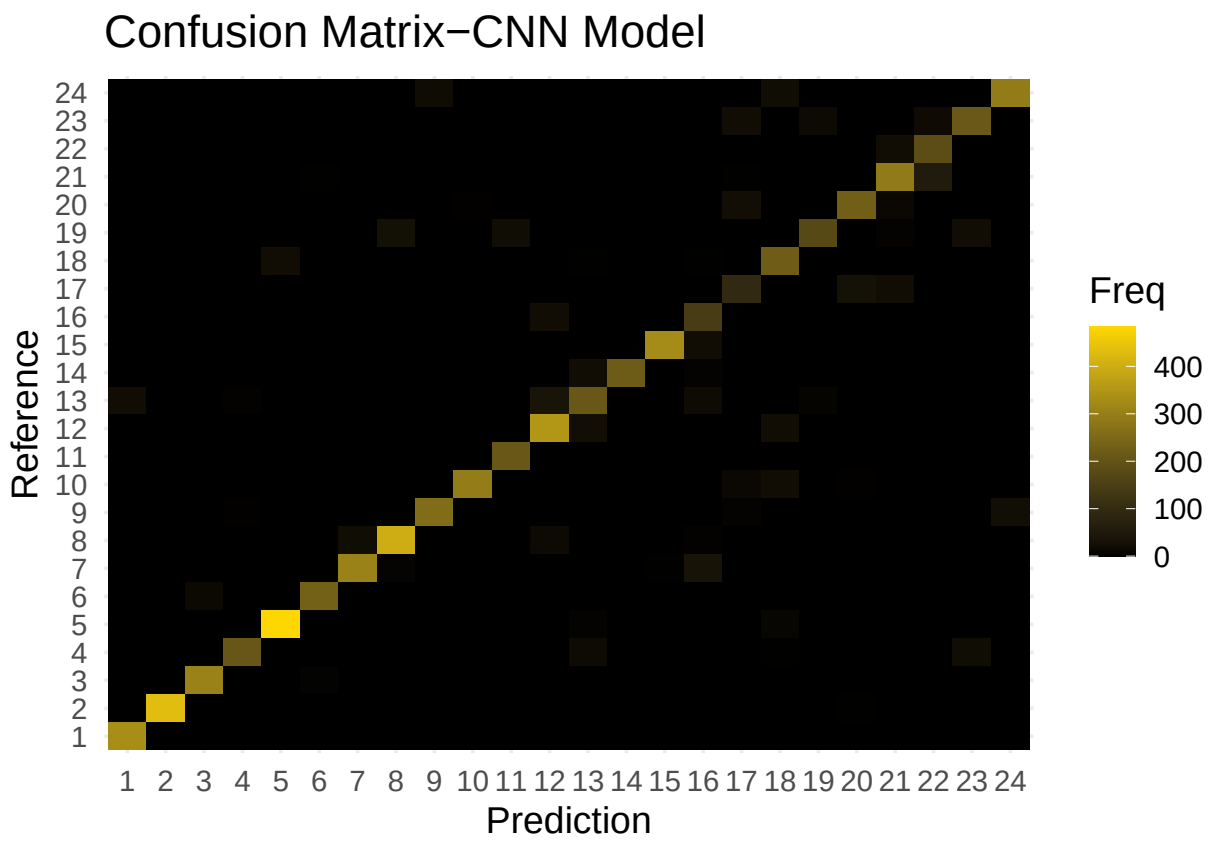


Figure 9: Confusion matrix for the CNN model on the test set, illustrating common misclassifications between visually similar ASL letters.

```
      Model TestAccuracy
```

1 LDA 0.4237312 2 QDA 0.1065254 3 KNN 0.8036810 4 Decision Tree 0.1694088 5 Bagging 0.7479085 6 Boosting 0.7617122 7 CNN 0.8933352

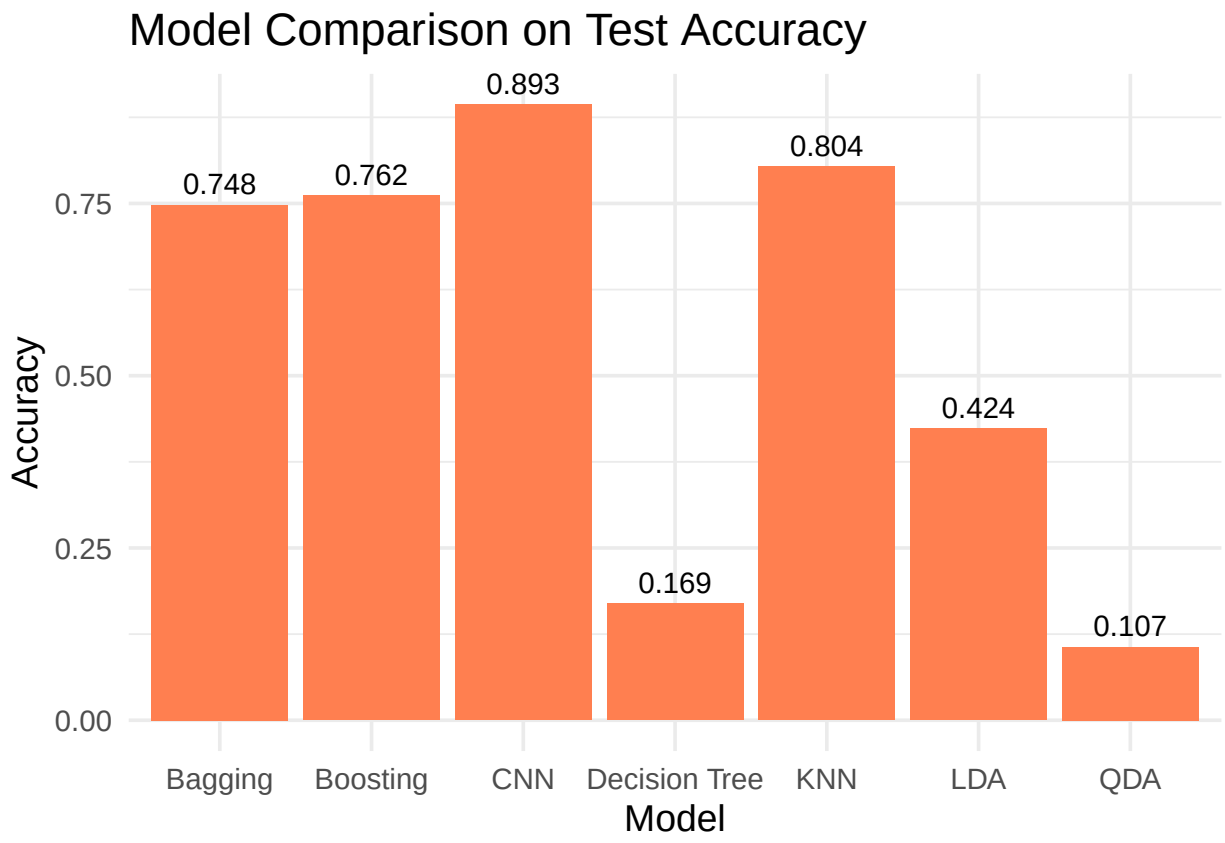


Figure 10: Comparison of test accuracy across all models, highlighting the superior performance of CNN and KNN relative to classical methods.